xbnh-gpu

XBeach non-hydrostatic on the GPU: installation, use and validation

Stefano Biondi

Department of Civil and Coastal Engineering, University of Florida

Version 2.0, October 2026

Abstract

xbnh-gpu is a CUDA implementation of XBeach (trunk revision 6167) in its non-hydrostatic (phase-resolving) mode, `wavemodel = nonh`. Version 2 covers every option that XBeach accepts in this mode: one-layer and reduced two-layer flow on 1D and 2D grids of any XBeach type, all boundary types, spectral wave boundaries, tide, wind, sources, vegetation, groundwater, ships, sediment transport and morphology including the gravel formulations of XBeach-G, and Fortran and netCDF output. It reads XBeach input decks unchanged and writes the same output files. On 96 feature decks, one per option, 1D runs agree with Fortran XBeach to round-off and 2D runs to the tolerance of the iterative pressure solver; with both solvers converged to 10^{-11}, 2D water levels agree to 10^{-13}. On a rip-channel beach of 0.16 to 8.7 million cells, one RTX 2000 Ada laptop GPU ran 2.0 to 3.3 times (double precision) and 3.9 to 7.5 times (single precision) faster than XBeach with MPI on a 24-core workstation. This guide explains how to install the code without administrator rights, build it for any recent NVIDIA GPU, run existing XBeach decks, and verify the results against a reference XBeach build that the repository also sets up.

Contents

1 What the code does **3**

2 Requirements **3**

3 Installation **4**
3.1 Automatic 4
3.2 Manual 4

4 Running a model **5**
4.1 Chaining segments 5
4.2 Output 5
4.3 Single or double precision 6
4.4 GPU-specific keys 6
4.5 Supported XBeach options 7

5 Reference XBeach and tests **7**
5.1 Feature suite 8
5.2 Quick tests 8

6 Validation **9**
6.1 Feature suite 9
6.2 Solver tolerance 9
6.3 Rip-channel beach 10
6.4 Cyclic-feature deck 10

7 Performance **11**

8 Numerical notes **12**
8.1 XBeach behaviours that are reproduced 12
8.2 Other notes 12

9 Troubleshooting **13**

10 License and citation **13**

1 What the code does

XBeach in non-hydrostatic mode (`wavemodel = nonh`) resolves individual waves with a depth-averaged momentum equation plus a vertical momentum equation and a non-hydrostatic pressure. The pressure is found from a Poisson equation at every time step. These equations are local stencil operations on a structured grid, which suit GPUs well: the cost is set by memory bandwidth, and a GPU has several times the bandwidth of a CPU socket.

xbnh-gpu follows the XBeach Fortran source routine by routine, each routine becoming one or more CUDA kernels:

- input, grid and initial state (`params.F90`, `spaceparams.F90`, `initialize.F90`);
- wet and dry cells and the time step (`wetcells.F90`, `timestep.F90`);
- boundary conditions, tide, wind, discharges and rainfall (`boundaryconditions.F90`, `compute_tide_zs0.F90`, `readwind.F90`, `discharge.F90`, `rainfall.F90`);
- spectral wave boundaries (`waveparamsnew.F90`, `spectral_wave_bc_module.F90`), with the phase-resolved sums and bound long waves on the GPU;
- momentum, viscosity and friction (`flow_timestep.F90`, `bedroughness.F90`);
- the non-hydrostatic predictor, breaking and pressure correction (`nonh.F90`), one-layer and reduced two-layer;
- second-order advection (`flow_secondorder.F90`);
- vegetation, groundwater, ships and drifters (`vegetation.F90`, `groundwater.F90`, `ship.F90`, `drifters.F90`);
- sediment transport, bed update and avalanching (`sed_transport.F90`, `morphevolution.F90`, `bedroughness.F90`, `interp.F90`);
- output (`ncoutput.F90`, `postprocess.F90`, `varianceupdate.F90`).

Where the Fortran depends on its sequential loop order, the GPU code reproduces the same result. This covers the neighbour checks of the breaking switch, loop variables that XBeach reuses from earlier loops, in-place sweeps such as avalanching (one GPU thread per grid row or column, in XBeach order), and single-precision constants in comparisons. Section 8 lists the XBeach behaviours that are reproduced on purpose.

There are two intentional differences.

1. **Pressure solver.** XBeach uses SIP, an incomplete-LU sweep that is sequential by construction. xbnh-gpu solves the same five-diagonal system with Jacobi-preconditioned BiCGStab to a relative residual of 10^{-3} (key `gpu_solver_acc`). In 2D the agreement with XBeach is set by the two solver tolerances: with both tightened to 10^{-11} (`solver_acc`, `gpu_solver_acc`) the codes agree to 10^{-13}. 1D grids use a direct tridiagonal solve (parallel cyclic reduction), as exact as the Fortran Thomas solve, and the same BiCGStab replaces SIP for the 2D groundwater head.
2. **Cyclic laterals.** XBeach supports `cyclic = 1` only under MPI, where the halo rows 1–2 and n_y–$n_y + 1$ are copied from the opposite side of the domain (period $n_y - 3$ rows). xbnh-gpu keeps the whole domain in one block, makes the same halo copies after every update of a prognostic field, and solves the pressure equation with periodic coupling. XBeach MPI does not exchange the velocities between the pressure correction and the continuity equation (section 8); xbnh-gpu does.

2 Requirements

- An NVIDIA GPU with compute capability 6.0 or newer: Pascal (P100, GTX 10xx) and later, including V100, T4, RTX 20xx/30xx/40xx, RTX Ada, A100 and H100. Check yours with `nvidia-smi --query-gpu=name,compute_cap --format=csv`.

- The NVIDIA driver, version 525.60 or newer (CUDA 12). On Windows, use WSL2 with the normal Windows NVIDIA driver; nothing GPU-related is installed inside WSL.
- Linux x86-64, or Windows 10/11 with WSL2 (Ubuntu). Installing WSL itself may need administrator rights once; everything after that runs in user space.
- About 3 GB of disk for the build environment, and 2 GB more for the optional reference XBeach.
- GPU memory: about 0.8 kB per grid cell in double precision and about half of that in single precision. An 8.7-million-cell grid used 6.8 GB in double precision.

No administrator rights are needed. The CUDA compiler, a compatible host C++ compiler, Python, and for the reference model gfortran and Open MPI, all come from conda-forge through micromamba. They are installed in the user's home directory.

3 Installation

3.1 Automatic

```
git clone <repository URL> xbnh-gpu
cd xbnh-gpu
./install.sh                    # GPU code only
./install.sh --with-xbeach      # GPU code and the reference Fortran XBeach (10-20
    min more)
source env.sh                   # in every new shell
bash tests/run_tests.sh
```

`install.sh` does the following.

1. It checks that `nvidia-smi` works and prints the GPU, its compute capability and the driver.
2. It installs micromamba in `~/.local/bin` if it is not already on the PATH. Environments go to `$MAMBA_ROOT_PREFIX`, which defaults to `~/micromamba`.
3. It creates the environment `xbnh-gpu` from `environment.yml`: CUDA 12.6 `nvcc`, GCC 13, make, Python 3.11 with NumPy, pandas and Matplotlib.
4. It builds `bin/xbnh_gpu` (double precision) and `bin/xbnh_gpu_sp` (single precision) for the GPU in the machine (`ARCH=native`).
5. With `--with-xbeach`, it also creates `xbnh-xbeach` (gfortran 13, Open MPI, autotools, Subversion) and runs `xbeach/build_xbeach.sh` (section 5).
6. It writes `env.sh`, which activates the environment, puts `bin/` on the PATH, and sets `XBEACH_SERIAL` and `XBEACH_MPI` to the reference executables.

3.2 Manual

With a CUDA toolkit (12.x) already available, for example on a cluster:

```
module load cuda                # on HPC systems; any CUDA >= 11.8 with a C++20
    host compiler
make ARCH=sm_80                 # cross-compile, e.g. on a login node without a
    GPU
```

ARCH	GPUs
native	the GPU in this machine (default)
sm_60	P100, GTX 10xx
sm_70	V100
sm_75	T4, RTX 20xx
sm_80	A100
sm_86	RTX 30xx, A40, A10
sm_89	RTX 40xx, RTX Ada, L4, L40
sm_90	H100

4 Running a model

xbnh-gpu runs in the folder that holds `params.txt`, like XBeach:

```
cd my_deck
xbnh_gpu          # double precision
xbnh_gpu_sp       # single precision
```

Use a separate folder from any XBeach run of the same deck. The output files have XBeach names and are overwritten. Linking the large input files avoids copies:

```
mkdir run_gpu && cd run_gpu && cp ../xbeach_run/params.txt .
for f in z.grd nhbcflist.bcf nh_series00001.bcf water_elev.dat wind.txt hotstart_
    *; do
  ln -s ../xbeach_run/$f .; done
xbnh_gpu > stdout.txt
```

4.1 Chaining segments

Long storms are usually run as a chain of shorter segments, each started from the state at the end of the previous one. With `writehotstart = 1` the GPU code writes the nine files that `hotstart = 1` reads (`hotstart_zs, zb, uu, vv, wb, ws, breaking, umean, vmean` plus a six-digit number) every `tinth` seconds, numbered from 1, in the same ASCII format as XBeach, so the files are interchangeable between the two codes. A 900 s segment written at its end is restarted with:

```
mkdir seg2 && cd seg2
cp ../seg1/hotstart_*000001.dat .        # copy, do not link: seg2 writes files
    of the same names
cp ../deck_seg2/params.txt ../deck_seg2/water_elev.dat ../deck_seg2/wind.txt .
python $XBNH/tools/set_params.py params.txt hotstart=1 hotstartfileno=1
    writehotstart=1 tinth=900 tstop=900
xbnh_gpu_sp > stdout.txt
```

Copy the hotstart files rather than linking them: a run that writes hotstart files of the same name would write through the links into the previous segment. Time restarts from zero in every segment, as in XBeach, so the tide, wind and boundary files of each segment start at its own $t = 0$.

Spectral wave boundaries. Since version 2 the GPU code synthesises the boundary series from the spectra itself (`wbctype = parametric, jonstable, swan, vardens`) and writes `nhbcflist.bcf` and the `nh_series` files as XBeach does, so a later segment can reuse them with `wbctype = reuse`. The sums over wave components, which take one to two hours on one CPU core for an offshore boundary of several thousand rows and a 900 s segment, run on the GPU. With `random = 0` the series are those of XBeach.

4.2 Output

Output follows `outputformat`, as in XBeach.

- `netcdf`: `xboutput.nc` with the dimensions, variables and attributes that XBeach writes (global, point, mean and drifter output, `_parameters`), `remdryoutput` and `outputprecision`. Build with netCDF (the default when `nc-config` is found).
- `fortran`: for each variable listed under `nglobalvar`, one file `<var>.dat`, one float64 record of $(n_x + 1)(n_y + 1)$ values (times the number of layers or fractions) per output time, with i (cross-shore) varying fastest; integer variables as int32. Read it in Python with `np.fromfile('zs.dat','<f8').reshape(-1, ny+1, nx+1)`. `dims.dat`, `xy.dat`, `<var>_mean.dat` (and `_var`, `_min`, `_max`) and `drifterNNN.dat` as in XBeach.
- Point output for each point under `npoints`: `<name>.dat`, one record per output time of `[t, var1, var2, ...]` in float64. Points snap to the nearest cell centre, and runup gauges follow the waterline, as in XBeach. Output times come from `tintg`, `tintp`, `tintm` or the lists `tsglobal`, `tspoints`, `tsmean`.
- `xbgpu_log.txt`: precision, number of steps, wall time, solver time and iterations, and cost per cell and step. Progress goes to standard output every 2000 steps.

4.3 Single or double precision

Single precision is about 2.2 times faster and uses about half the memory. In the hydrodynamic tests it gave the same statistics as double precision; bed changes differ more (section 6). Consumer GPUs are much slower in double precision than in single; data-centre GPUs (A100, H100) are not.

4.4 GPU-specific keys

Key	Default	Meaning
`gpu_solver_acc`	0.001	relative residual of the BiCGStab pressure solve (2D) and of the 2D groundwater head solve
`gpu_maxit`	200	maximum BiCGStab iterations per step (about 13 are typical)
`gpu_hotstart_keepuv`	1	keep `uu, vv` from the hotstart files. Trunk XBeach zeroes them after reading; 0 reproduces that. The reference build here keeps them (section 5).

XBeach ignores these keys, apart from a warning about unknown parameters.

4.5 Supported XBeach options

Group	Supported
Grid	1D (`ny = 0`) and 2D ($n_y \geq 3$); `vardx = 0` or 1 (`xfile`, `yfile`); `alfa`; `gridform = delft3d` (`xyfile`); `posdwn`, `xori`, `yori`, `thetanaut`
Flow	one layer, or reduced two-layer (`nonhq3d = 1`, `nhlay`); `nhbreaker = 0, 1` (2, 3 with XBeach-G); `maxbrsteep`, `reformsteep`, `breakviscfac`; `dispc`, `Topt`, `kdmin`; `secorder`, `oldhu`, `umin`, `eps`, `epsi`, `hmin`, `CFL`, `maxdtfac`; `viscosity`, `smag`, `nuh`, `nuhv`; Coriolis (`lat`, `wearth`); `maxfacg`
Friction	`bedfriction = chezy, cf, manning, white-colebrook, white-colebrook-grainsize`; uniform `bedfriccoef` or per-cell `bedfricfile`; `maxcf`, `mincf`; `friction_acceleration` (`mccall`, `nielsen`), `friction_turbulence`, `dynamicroughness`
Offshore	`front = nonh_1d`, `ARC = 0` or 1; `wbctype = params`, `ts_nonh` (U, V, Z, W, dU, dV, Q columns), `reuse` (`nhbcflist.bcf`, several entries), `off`, and the spectral types `parametric`, `jonstable`, `swan`, `vardens` (`FILELIST`, `LOCLIST`, `random`, `order = 1, 2`, `highcomp`, `nonhspectrum`)
Back	`back = wall`, `abs_1d`, `abs_2d`
Laterals	`left`, `right` = `neumann`, `wall`, `no_advec`, `neumann_v`, `abs_1d`; or `cyclic = 1`
Water level	`zs0`, `zs0file` with `tideloc = 0, 1, 2, 4`, `paulrevere`, `tidetype = velocity, instant`; `zsinitfile`
Sources	wind (`windv`, `windth`, `windfile`); point and line discharges; rainfall (constant or file)
Vegetation	`vegetation = 1` (several species and sections, bulk drag with $C_d < 0$, `veguntow`); porous in-canopy model `porcanflow`
Groundwater	`gwflow = 1`: hydrostatic or non-hydrostatic head (`gwnonh`, `gwheadmodel`), laminar or turbulent (`gwscheme`), `gwhorinfil`, `friction_infiltration`, aquifer and head files
Ships	`ships = 1`: ship pressure fields along prescribed tracks, with or without heading, and ship forces
Sediment	`sedtrans`, `morphology`; `form = soulsby_vanrijn, vanthiel_vanrijn, vanrijn1993, nielsen2006, mccall_vanrijn, intrasedtr`; several fractions (`ngd`) and bed layers (`nd`); `lwt`, `sourcesink`, `fallvelred`, `dilatancy`, `bermslopetransport`, bed-slope effects; avalanching; `struct`, `ne_layer`; `morfac`, `morfacopt`, `morstart`; `setbathy`; `useXBeachGSettings`
Restart	read: `hotstart = 1`, `hotstartfileno`; write: `writehotstart = 1` every `tinth` or at `tshotstart` (XBeach ASCII format, files numbered from 1), including sediment and groundwater state
Output	`outputformat = fortran, netcdf, debug`; global (grid and per-ship variables such as `shipFx`), point, runup-gauge, mean (`nmeanvar`) and drifter output; `tintg`, `tintp`, `tintm` or `tsglobal`, `tspoints`, `tsmean`; `rotate`; `remdryoutput`, `outputprecision`

Not supported: wave-action models (`wavemodel` other than `nonh`); options that need short-wave forcing (`swrunup`, `tidetype = hybrid`, the quasi-3D sediment profile `nz > 1`); `gwfastsolve = 1`, which reads an uninitialised index in XBeach r6167; `nglobalvar = 999`; and more than one GPU. Unsupported options stop the program with a message instead of being ignored. The transport formulas `mpm`, `wilcock_crow`, `engelund_fredsoe`, `wong_parker`, `fl_vb` and `fredsoe_deigaard` are accepted by XBeach r6167 but never called, so they give no transport in either code.

5 Reference XBeach and tests

`xbeach/build_xbeach.sh` downloads XBeach trunk r6167 from the Deltares Subversion server (anonymous access) and applies two small patches (`xbeach/patch_xbeach.py`). It then builds `xbeach/dist_serial/bin/xbeach` and `xbeach/dist_mpi/bin/xbeach` (Open MPI), with Fortran output only. The two patches are:

1. **Hotstart velocities** (`initialize.F90`). Trunk reads `uu, vv` from the hotstart files and then sets them to zero in `flow_init`, so every restart starts from rest. The patch keeps them.

2. **Point output** (`postprocess.F90`). A missing comma in a log-message format makes runs with output points stop at start-up with "Missing comma between descriptors". This happens always in the serial build and intermittently under MPI. Only the message format changes.

`cyclic = 1` needs the MPI build with at least two ranks: `mpirun -np 4 $XBEACH_MPI`.

5.1 Feature suite

`tests/run_features.sh` builds one small deck per XBeach option (96 decks, listed in `tests/features/suite.txt` and generated by `examples/features/make_case.py`), runs them on the GPU and checks the global output:

```
bash tests/run_features.sh                  # GPU only, against stored XBeach
    statistics, about 5 min
bash tests/run_features.sh --xbeach -j 8    # also runs every deck in Fortran
    XBeach, 8 at a time
bash tests/run_features.sh sed2d gw_nh2d    # selected decks
```

The base deck is a barred beach 520 m long at $\Delta x = 2$ m with 120 s of irregular waves ($H_{m0} = 1$ m, $T_p = 8$ s); 2D decks have 61 alongshore rows. With `--xbeach` the GPU output is compared with the XBeach output field by field over $t \geq 40$ s: relative RMS differences of H_s (`Hs_rel`), of the mean current (`current_rel`), of the final bed (`bed_rel`) and of the sediment concentration (`cc_rel`), and the RMS difference of the instantaneous water level (`zs_rms_m`). Every other output file (extra global variables, point and runup-gauge files, mean files, drifters, per-ship forces) is compared as well, by its largest difference relative to its largest value (`files_rel`). The limits are 10^{-6} for 1D decks, which agree with XBeach to round-off, and 2×10^{-3} (H_s) and 10^{-2} (current, bed, concentration) for 2D decks, whose agreement is set by the pressure-solver tolerance; `files_rel` has a 2D limit of 0.1, because a maximum over all cells and times picks up cells at the waterline that are wet in one code and dry in the other. Without `--xbeach` the deck statistics (mean H_s, RMS mean current, mean water level, RMS bed change, mean concentration) are compared with the XBeach values in `tests/features/reference.json`, with the same limits.

5.2 Quick tests

`tests/run_tests.sh` runs two small decks on the GPU in both precisions:

- a rip-channel beach (39 k cells, 120 s);
- the cyclic-feature deck (40 k cells, 100 s; cyclic laterals, `abs_2d`, vector boundary, tide and wind files, point output);
- a hotstart round trip on the cyclic-feature deck: a 40 s run writes a hotstart, a second run restarts from it, and the restarted water level and velocities must match to 10^{-6};
- a per-cell Manning map: a uniform `bedfricfile` of 0.02 must reproduce `bedfriccoef = 0.02` to round-off (10^{-9}).

When the reference XBeach exists, the same decks run in Fortran XBeach and the agreement is checked against thresholds. Otherwise the GPU statistics are checked against values stored in `tests/reference.json`. The thresholds are several times the differences that XBeach itself shows between rank counts. The tests take two to five minutes. A passing run with the reference XBeach ends like this (the numbers are from the RTX 2000 Ada):

```
Fortran XBeach (serial) against GPU
  Hs_rel          0.0014  (limit 0.01)  ok
  current_rel     0.0045  (limit 0.03)  ok
  zs_rms_m        0.0027  (limit 0.01)  ok
...
Fortran XBeach (MPI, 2 ranks) against GPU
  Hs_rel          0.0032  (limit 0.02)  ok
```

```
   current_rel     0.0046  (limit 0.1)  ok
   zs_rms_m        0.0029  (limit 0.02)  ok
== 3. hotstart write and restart (cyclic-feature deck)
 ...
   zs  max difference after restart 5.0e-09  (limit 1e-06)  ok
   uu  max difference after restart 5.0e-09  (limit 1e-06)  ok
   vv  max difference after restart 5.0e-09  (limit 1e-06)  ok
   PASS
== 4. Manning map (bedfricfile): a uniform map must reproduce bedfriccoef
 ...
   zs  max difference 9.6e-13  (limit 1e-09, round-off)  ok
   PASS
tests passed: 12, failed: 0
```

6 Validation

6.1 Feature suite

The 96 decks of the feature suite (section 5.1) were run in Fortran XBeach r6167 (serial) and in xbnh-gpu on an RTX 2000 Ada. All pass. Table 1 gives the largest and median differences over the decks.

Table 1: Feature suite: differences between xbnh-gpu (double precision) and XBeach over 43 1D and 53 2D decks.

Metric	1D largest	1D median	2D largest	2D median
H_s, relative RMS	5.5×10^{-15}	1.5×10^{-15}	4.9×10^{-4}	1.4×10^{-5}
Mean current, relative RMS	4.2×10^{-15}	9.8×10^{-16}	7.2×10^{-4}	4.4×10^{-5}
Water level, RMS (m)	3.4×10^{-15}	1.4×10^{-15}	3.7×10^{-4}	2.4×10^{-5}
Bed level, relative RMS	5.5×10^{-13}	9.2×10^{-14}	2.8×10^{-4}	8.1×10^{-5}
Concentration, relative RMS	9.0×10^{-15}	7.4×10^{-15}	9.4×10^{-5}	5.8×10^{-5}

In single precision the 1D hydrodynamics agree to 2×10^{-6} and the 2D decks as well as in double precision, but the bed change after 120 s differs by 0.5 % in the median deck and by up to 2.5 %. Use double precision for morphological runs.

6.2 Solver tolerance

The 2D sediment deck of the suite was run in both codes with `solver_acc` and `gpu_solver_acc` set to the same value. The largest difference over the run, relative to the largest value of each field, falls in proportion to the tolerance (Table 2): the 2D differences are those of the two iterative solvers, not of the implementation.

Table 2: 2D sediment deck: largest difference between the codes, relative to the largest value.

Tolerance (both codes)	z_s	u	c	z_b
defaults (5×10^{-3}, 30 it.; 10^{-3})	5.3×10^{-3}	2.1×10^{-3}	3.1×10^{-4}	1.9×10^{-6}
10^{-3}	1.8×10^{-3}	9.1×10^{-4}	2.1×10^{-4}	4.8×10^{-8}
10^{-5}	3.8×10^{-7}	2.1×10^{-7}	2.3×10^{-8}	3.4×10^{-11}
10^{-7}	3.2×10^{-9}	2.6×10^{-9}	2.1×10^{-10}	4.0×10^{-13}
10^{-9}	3.6×10^{-11}	1.6×10^{-11}	1.3×10^{-12}	2.2×10^{-15}
10^{-11}	3.3×10^{-13}	1.2×10^{-13}	9.3×10^{-15}	1.1×10^{-16}

The subsections below are the deck-level validations of version 1; the version-2 code reproduces them (rip-channel beach: H_s 0.01 %, mean current 0.01 %, z_s 0.13 mm against serial XBeach, because the time step now lands on the output times as in XBeach).

6.3 Rip-channel beach

The deck has 39 k cells and dx = 2 m (`examples/rip_beach`):

- a 1:50 slope with a shore-parallel bar cut by rip channels;
- JONSWAP waves (H_{m0} = 1 m, T_p = 8 s) driven through `boun_U.bcf`, for 200 s.

Statistics are for $t \geq 60$ s (Table 3, Figure 1).

Table 3: Rip-channel beach: differences against serial Fortran XBeach.

Metric	GPU fp64	GPU fp32	XBeach MPI-4
H_s map, relative RMSE	0.16 %	0.18 %	0.36 %
Mean current, relative RMSE	0.62 %	0.61 %	1.59 %
Set-up RMSE	0.21 mm	0.21 mm	0.43 mm
Instantaneous z_s, RMS difference	3.5 mm	3.6 mm	4.1 mm

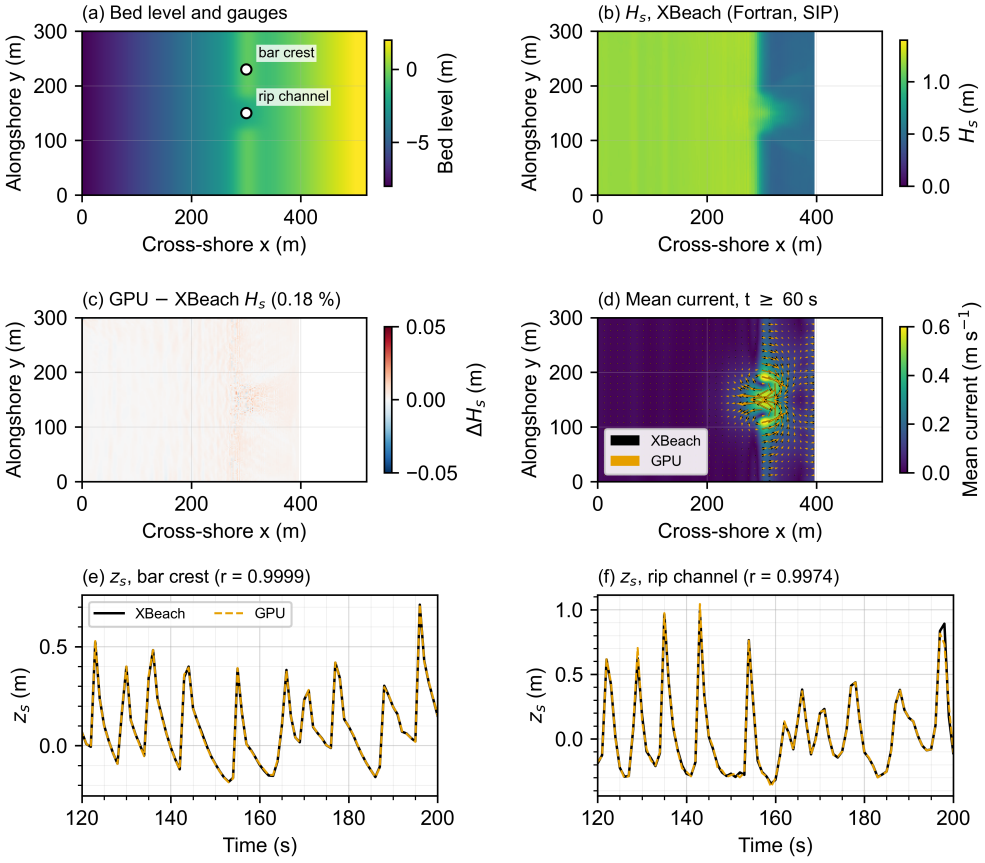


Figure 1: Rip-channel beach. (a) Bed and gauges; (b) XBeach H_s; (c) GPU minus XBeach H_s; (d) mean currents of both codes; (e, f) water level at the bar crest and in the rip channel.

6.4 Cyclic-feature deck

`examples/cyclic_features` combines the options of a field-scale storm deck on 40 k cells:

- cyclic laterals;
- a wet `abs_2d` back boundary;
- a short-crested sea, 20° oblique, given as a vector U, V, Z file;

- a rising tide from `zs0file` and a time-varying wind;
- point output, and a hotstart written at 100 s.

Table 4 lists the results.

Table 4: Cyclic-feature deck (relative RMSE of maps; z_s as RMS difference).

Comparison	H_s	Mean current	Longshore current	z_s
Serial XBeach vs GPU, hotstart, Neumann laterals	0.20 %	0.82 %	1.2 %	2.6 mm
XBeach MPI-2 vs GPU, cyclic	0.34 %	2.0 %	2.3 %	4.7 mm
XBeach MPI-4 vs GPU, cyclic	0.54 %	3.4 %	4.9 %	6.2 mm
XBeach MPI-4 vs MPI-2 (yardstick)	0.44 %	2.8 %	2.9 %	5.5 mm

7 Performance

Table 5 lists the wall time per cell and time step on an RTX 2000 Ada (16 GB) and an i9-14900K (8 performance and 16 efficiency cores). The deck is the rip-channel beach, stretched alongshore. Figure 2 shows the same benchmark.

Table 5: Cost per cell and time step (s).

Cells	XBeach MPI, best rank count	GPU fp64	GPU fp32	Speed-up fp64 / fp32
0.16 M	8.8e-8 (24 ranks)	4.5e-8	2.2e-8	2.0 / 3.9
0.63 M	1.0e-7 (32 ranks)	4.5e-8	1.9e-8	2.3 / 5.3
2.5 M	1.35e-7 (32 ranks)	4.3e-8	1.9e-8	3.2 / 6.9
8.7 M	1.39e-7 (24 ranks)	4.2e-8	1.9e-8	3.3 / 7.5

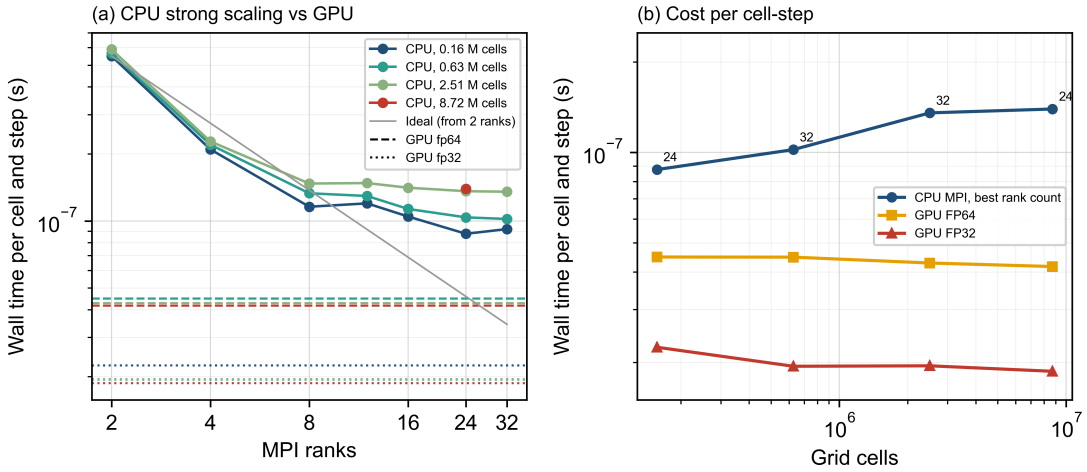


Figure 2: (a) XBeach MPI strong scaling on one workstation (lines) against the GPU (horizontal: dashed fp64, dotted fp32); (b) cost per cell and step against grid size.

The scaling has three features worth knowing.

- The CPU stops gaining beyond about 8 ranks, because all cores share the same memory bandwidth.
- The GPU cost per cell is flat from 0.16 to 8.7 million cells.
- The pressure solve takes about half of the GPU time in fp64 and two thirds in fp32.

A data-centre GPU (A100, H100) is expected to be another 8 to 15 times faster in double precision, but this has not been measured yet.

To reproduce the benchmark:

```
SIZES="600 2400 9600" RANKS="2 4 8 16" bash tools/benchmark.sh bench
python tools/plot_benchmark.py bench/benchmark.csv
```

8 Numerical notes

8.1 XBeach behaviours that are reproduced

Verification against the Fortran code exposed behaviours of XBeach r6167 that change results. xbnh-gpu reproduces them, so that an existing deck gives the XBeach answer, except where the behaviour is a crash or reads undefined memory.

Area	XBeach r6167 behaviour
`ts_nonh` boundary	The first call of the boundary routine opens `boun_U.bcf` and then advances it, so the first step interpolates between records.
Cell-centre v_e	Lags one flow step (computed before the face velocity is updated); used by output and the canopy model.
1D wind	With `ny = 0` the x-directed wind also enters the y-component of the stress, so the stress is $\sqrt{2}$ too large.
Transport formulas	`mpm`, `wilcock_crow`, `engelund_fredsoe`, `wong_parker`, `fl_vb` and `fredsoe_deigaard` are accepted but never called: no transport.
Bed load	The bed-load fluxes of the sand formulas are zero in non-hydrostatic mode.
`intrasedtr`	The suspended-load adaptation time is never set: bed load only.
`nielsen2006`	XBeach stops with a segmentation fault unless `streaming = 1`; xbnh-gpu runs with a warning.
Hotstart	Trunk XBeach sets the velocities read from the hotstart files to zero; `gpu_hotstart_keepuv = 0` reproduces this (default 1 keeps them, as the patched reference build does).
Placeholder defaults	Keys that a deck does not set can keep the value -123 and enter the computation, for example `tintm` in the weight of the first averaging interval.
2D avalanching	With a time-series boundary `Hrms` is not read and stays at -123, so the swash excursion that scales the alongshore avalanching flux is undefined; wherever an alongshore slope exceeds the critical slope the whole erodible layer moves in one step and the run fails. Set the boundary with `wbctype = params` or a spectrum, or avoid alongshore slopes above `wetslp`.
`highcomp`	The second-order components take their phases from the frequency bin above each component, a row is cleared when $k_3 h > 5$ at its first entry, and no alongshore flux is added.
`gwfastsolve`	Fills its matrix with an uninitialised index; not supported.
Cyclic laterals (MPI)	Velocities are not exchanged after the pressure correction (below); not reproduced.

8.2 Other notes

- **2D agreement and the solver tolerances.** Both codes stop their pressure iterations when the residual falls below a tolerance times the norm of the right-hand side, but SIP and BiCGStab reach different errors at the same residual. On steps with a sharp local forcing, such as the first step of a ship pressure field, BiCGStab needs a residual two orders of magnitude smaller than SIP for the same accuracy: on the ship test deck with waves, both codes at 10^{-11} differ by 10^{-6}, and with `gpu_solver_acc = 1e-13` they agree to 10^{-14}.
- **Results do not depend on the decomposition.** The GPU domain is a single block, so results do not change with core count. XBeach MPI results do: between 2 and 4 ranks, mean currents differ by about 3 % in the tests above.

- **One cause of the XBeach drift.** XBeach MPI exchanges `uu, vv` after the momentum equation, the predictor and the second-order advection. It does not exchange them after the pressure correction, so at each subdomain seam the continuity equation uses an uncorrected halo velocity. Hotstart files written by XBeach MPI on a field-scale deck show this: their halo rows of `vv` differ from the rows they copy by up to 4.6 cm/s, while `zs` matches exactly.
- **Run-to-run reproducibility.** Two runs of xbnh-gpu on the same GPU agree to about 10^{-14} m, not bit for bit, because the dot products of the solver are summed with atomic operations.
- **Unstable corner with Neumann laterals.** With Neumann laterals and an oblique vector boundary, the corner where the offshore and lateral boundaries meet can become unstable in both codes. After that, the runs separate. Use cyclic laterals for oblique seas.

9 Troubleshooting

`nvidia-smi` not found, or "CUDA driver version is insufficient"
Install or update the NVIDIA driver (525.60 or newer). On WSL2, update the Windows driver; do not install a Linux driver inside WSL.

`nvcc fatal: Unsupported gpu architecture 'compute_native'`
No GPU is visible during the build (for example on a login node). Build with `make ARCH=sm_XY` from the table in section 3.

"CUDA error out of memory"
The grid is too large for the card. Use `xbnh_gpu_sp` (about half the memory) or a larger GPU.

"xbnh_gpu: ... not supported"
The deck uses an option outside the supported list; the message names it.

"time step collapsed"
The solution became unstable and the time step went to zero; XBeach stops in the same way. Check the deck in XBeach first.

`micromamba create` fails or hangs (blocked conda-forge)
Some institutional networks block or inspect traffic to conda-forge. Set the proxy first (`export https_proxy=http://proxy:port`), or skip micromamba if a CUDA toolkit (12.x, with `g++` 10 or newer) is already installed: `make NVCC=/usr/local/cuda/bin/nvcc`, then run the tests with any Python that has NumPy. `env.sh` is then not needed; put `bin/` on `PATH` and set `XBNH` to the repository folder.

No MPI reference XBeach ("no XBeach MPI build found")
The tests then compare the cyclic-feature deck with stored XBeach statistics instead, which is enough to validate the GPU build. The MPI build needs Open MPI from `environment-xbeach.yml`.

XBeach stops with "Missing comma between descriptors"
This is an unpatched XBeach with output points. Use the build from `xbeach/build_xbeach.sh`.

10 License and citation

XBeach is developed by Deltares, UNESCO-IHE and Delft University of Technology and distributed under the GNU Lesser General Public License. xbnh-gpu translates XBeach routines to CUDA and is distributed under the same license (LGPL-3.0). Please cite the XBeach model [1] and its non-hydrostatic breaking formulation [2] when using xbnh-gpu.

References

[1] Roelvink, D., Reniers, A., van Dongeren, A., van Thiel de Vries, J., McCall, R., Lescinski, J. (2009). Modelling storm impacts on beaches, dunes and barrier islands. *Coastal Engineering*, 56(11–12), 1133–1152. doi:10.1016/j.coastaleng.2009.08.006

[2] Smit, P., Zijlema, M., Stelling, G. (2013). Depth-induced wave breaking in a non-hydrostatic, near-shore wave model. *Coastal Engineering*, 76, 1–16. doi:10.1016/j.coastaleng.2013.01.008